דו”ח סיכום — האקתון RISC-V
מאיץ Smith-Waterman על מעבד RISC-V + FPGA

עידן קיינן · ת.ז. 209703107 | נדים טלי · ת.ז. 207883372

*** 4 עמודים לכל היותר ***

×89.8	12,881	0 / 0	+2.837ns
האצה כוללת מול ה-baseline	מחזורים · מול 1,157,252 בתוכנה	DSP / Block RAM נוספים	WNS · 0 הפרות תזמון

1 מבוא

תיאור הבעיה ורקע

יישור רצפי DNA באלגוריתם Smith-Waterman עם Affine-Gap הוא חישוב כבד וחזרתי בגנומיקה (זיהוי דמיון, מוטציות והרכבת גנומים). נתון רצף query ומאגר של 8 רצפי reference (16 בסיסים A/C/G/T כל אחד), ויש לחשב לכל reference את ציון ה-local alignment הטוב ביותר. הציונים הצפויים: 26, 26, 23, 23, 24, 23, 23, 2.

מטרות

להאיץ את משימת החישוב על מעבד RISC-V בשילוב מאיץ FPGA — מזעור זמן הביצוע (קריטריון שיפוט ראשון) ושימוש מינימלי בחומרה (קריטריון שני), תוך שמירה על תוצאות זהות בדיוק לתוכנית המקורית ועל אפס הפרות תזמון.

הדרך והאמצעים

פיתוח מדורג ובטוח: (1) נקודת אפס — הרצת המקור; (2) אופטימיזציית תוכנה — rolling buffer וקידוד 2-ביט; (3) מאיץ חומרה ייעודי שמנצל את ממשק המאיץ מתרגיל ההכנה; (4) אינטגרציה ואימות. בכל שלב נשמרה גרסה עובדת ובוצע אימות מול תוצאת הייחוס.

2 תיאור הפתרון

מאיץ Smith-Waterman affine-gap ייעודי, ממופה-זיכרון (בסיס 0x80001300), המחשב alignment שלם של ה-query מול reference יחיד ומחזיר best_score. הליבה סדרתית — תא DP אחד במחזור — עם rolling buffer (שמירת שורה קודמת + מספר רגיסטרים, זיכרון O(n)), datapath signed 8-bit עם חיבורים רוויים (saturating, NEG_INF=−64), ו-FSM: IDLE→INIT→CELL→NEXT_ROW→DONE. הקלט מקודד 2-ביט לבסיס (A=0, C=1, G=2, T=3), כך שרצף שלם נכנס במילת 32 ביט אחת. **אין כפל בכלל.**

מה הואץ בחומרה

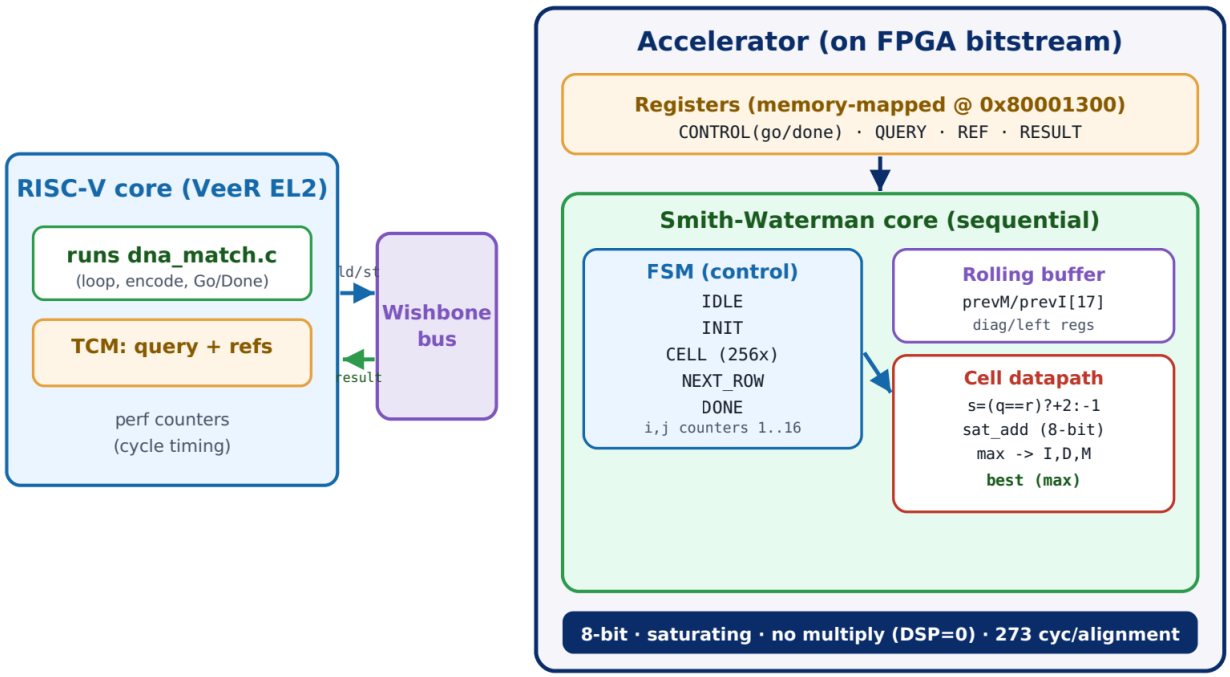
כל הלולאה הפנימית הכבדה — 256 תאי ה-DP לכל יישור: חישוב ציון ההתאמה s, הרקורסיות של I/D/M (חיבורים ו-max בלבד), ומעקב ה-best. בחומרה תא מחושב במחזור אחד עם השכנים ברגיסטרים, ללא גישות זיכרון.

מה נשאר בתוכנה (RISC-V)

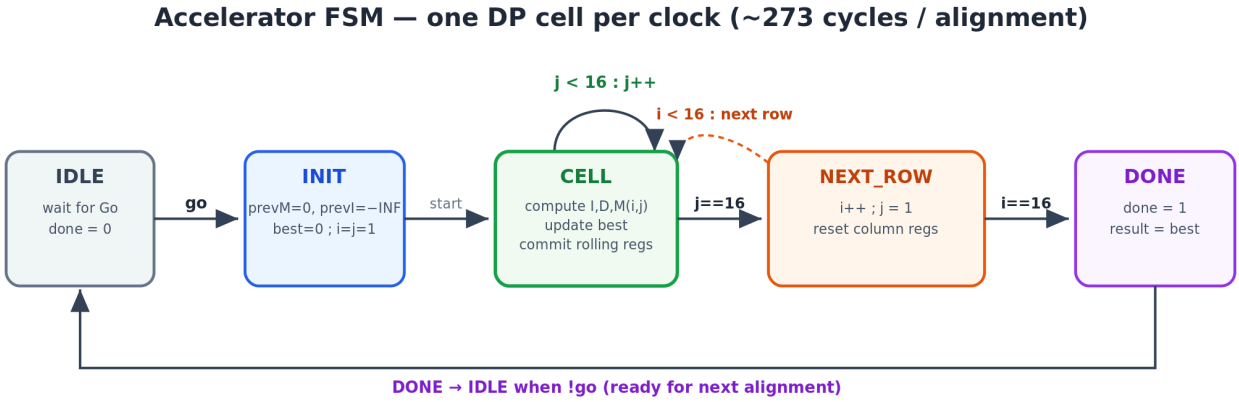
הלולאה החיצונית על 8 ה-references, קריאת הקלט מזיכרון ה-TCM, הקידוד ל-2-ביט, ה-handshake מול המאיץ (Go/Done), וההדפסה. הקלט/פלט ומוני הזמן נותרו על המעבד כנדרש בכללים.

בחירת מיקרו-ארכיטקטורה — סדרתי מול מערך סיסטולי

בחרנו במכוון במנוע סדרתי (תא DP אחד למחזור, ~273 מחזורים ליישור) ולא במערך סיסטולי (systolic array) המחשב אנטי-דיאגונל שלם של תאים במקביל — מבנה שהיה מקצר את הליבה לכ-2N−1≈31 מחזורים. זו החלטה מונחית-מדידה ולא תיאורטית: בשלב 5 בדקנו batch (Go אחד ל-8 יישורים) וקיבלנו דווקא תוצאה איטית יותר — 13,925 מול 12,881 מחזורים. הממצא הצביע שצוואר הבקבוק כבר עבר מהחישוב אל תקורת ה-CPU/באס (כ-**83%** מהזמן הכולל). לפי חוק אמדל, האצת הליבה (רכיב שכבר זניח בתקציב הזמן) כמעט לא הייתה משפרת את ה-90× הכולל, ובמקביל הייתה מגדילה משאבים ומסכנת את סגירת התזמון. לכן בחרנו במנוע הסדרתי: פשוט, חסכוני (**0 DSP**), קל לאימות, וסוגר תזמון בנוחות (WNS +2.837ns).



איור 1. דיאגרמת בלוקים של ארכיטקטורת הפתרון — RISC-V (VeeR EL2), אפיק Wishbone, ומאיץ Smith-Waterman (FSM · rolling buffer · cell datapath).



איור 2. מכונת המצבים (FSM) של המאיץ — תא DP אחד בכל מחזור; 16×16 + תקורה ≈ 273 מחזורים ליישור שלם.

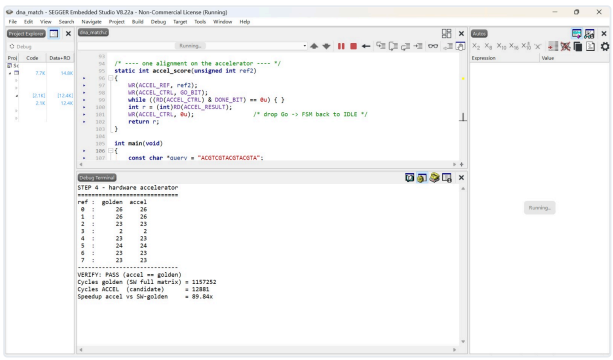
3 סביבת הפיתוח ותוצאות הפרויקט

סביבת הפיתוח

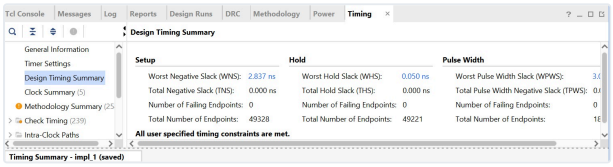
חומרה — Vivado 2023.2 (סינתזה, behavioral simulation, ILA) על Nexys A7 (xc7a100t). תוכנה — SEGGER Embedded Studio for RISC-V (קומפילציית C, J-Link, Debug Terminal). שפות: SystemVerilog ל-RTL ו-C ל-driver. אימות עזר: מודל Python מחזור-מדויק והרצת gcc על המארח.

תוצאות ההאצה

מדד	ערך
Baseline (תוכנה, מטריצה מלאה)	1,157,252 cycles
מאיץ חומרה (שלב 4)	12,881 cycles
האצה	**×89.8**
Timing	WNS +2.837 ns · 0 viol.
Block RAM / DSP נוספים	0 / 0
נכונות	8/8 · random 800/800



איור 3. פלט ה-Debug Terminal (שלב 4): self-check ש-accel==golden, 8/8 ציונים תואמים, ×89.84.



איור 4. Vivado Design Timing Summary: WNS +2.837ns, 0 endpoints כושלים — "All user specified timing constraints are met."

כיצד בדקנו את התוצאות

נכונות — הציונים נבדקים מול ה-golden (התוכנית המקורית, שהיא המפרט) בכל רמה: ב-Python (3,000 קלטים אקראיים, 0 שגיאות), בסימולציית xsim של Vivado (RESULT: PASS), ועל החומרה עצמה — self-check ש-accel==golden באותה הרצה, ובדיקת random של 800 קלטים אקראיים (0 אי-התאמות).

ביצועים — נמדדו ע"י מוני הזמן המובנים (pspMachinePerfCounter) שלא נגענו בהם, סביב אזור החישוב בלבד. אי-אפשר "לזייף" מהירות, כי ה-self-check של הנכונות רץ באותה הרצה.

4 סיכום

תרומת ההאצה

קיבלנו האצה של פי **~90** (`1,157,252 → 12,881` מחזורים) עם תוספת חומרה זעירה (`0 DSP, 0 Block RAM` ו-0 הפרות תזמון — תוצאה חזקה בשני קריטריוני השיפוט יחד. ה-`latency` דטרמיניסטי (273 מחזורים ליישור) ובלתי-תלוי בנתונים, כך שההאצה חסינה לכל קלט.

חשיבה ביקורתית — מה היה טוב

ניצול חכם של ממשק המאיץ מתרגיל ההכנה (שינוי קובץ RTL אחד בלבד); datapath צר וללא מכפלים (`DSP=0`); אימות רב-שכבתי ובלתי-תלוי (Python, סימולציה, חומרה); ופיתוח מדורג עם גרסה עובדת ובת-הגשה בכל שלב.

מה ניתן לשפר

הצוואר כעת הוא תקורת ה-CPU/באס (~83% מהזמן), לא החישוב. ניסינו `batch` (Go אחד ל-8) אך הוא לא שיפר — מה שאישש שה-handshake אינו הצוואר. שיפור עתידי: הקטנת תקורת התקשורת/קידוד, או הקבלה (`wavefront`) להורדת ה-273 מחזורי הליבה. בנוסף, הפתרון מכוונן לאורך 16; הרחבה לאורכים אחרים דורשת פרמטריזציה (קלה) של רוחב ה-datapath והמונים.